

Internet Relay Chat Class Project

Table of Contents

1. Introduction	3
2. Basic Information	3
3. Conventions	4
4. Message Infrastructure	4
4.1. Generic Message Format	4
4.1.1. Message Format	4
4.1.2. Definitions	5
4.1.3 Operation Codes (header.operation_code)	5
4.1.4 Restrictions	5
4.2. Operation Messages	6
4.2.1. Message Format	6
4.2.2. Definitions	6
4.2.3. Operation Codes (operation_message.operation_code)	6
4.2.4. Restrictions	7
4.3. Error Messages	7
4.3.1. Message Format	7
4.3.2. Definitions	7
4.3.3. Error Codes (error_message.err_code)	8
4.3.4. Restrictions	8
4.4. Global Message Restrictions	9
5. Client Messages	9
5.0.1 General Global Guidelines	9
5.1. Connecting to the Server	9
5.1.1 Definition	9
5.1.2 Usage	10
5.2. Create a Room	10
5.2.1 Definition	10
5.2.2 Usage	10
5.3. Join a Room	10
5.3.1 Definition	10
5.3.2 Usage	10
5.4. Leave a Room	11
5.4.1 Definition	11

5.4.2 Usage	11
5.5. Disconnect from the Server	11
5.5.1 Definition	11
5.5.2 Usage	11
5.6. Send a Message	11
5.6.1 Definition	11
5.6.2 Usage	12
5.7. Send a File	12
5.7.1 Definition	12
5.7.2 Usage	12
6. Server Messages	13
6.0.1 General Global Guidelines	13
6.1. List Members	13
6.1.1. Definition	13
6.1.2. Usage	13
6.2. List Rooms	14
6.2.1. Definition	14
6.2.2. Usage	14
6.3. Success	14
6.3.1. Definition	14
6.3.2. Usage	14
7. Error Handling	14
8. Extra Functionality	15
8.0. Extra functionality foreword	15
8.1. Private Messaging	15
8.2. Secure messaging	15
8.3. File Transfer	16
8.4. Cloud Connected Server	16
9. Security	16
10. Conclusion and Future Directions	16

1. Introduction

This specification outlines the scope of a custom Internet Relay Chat (IRC) application/protocol. The basis of the IRC application is that clients can join a server and choose to connect, create, or leave different rooms, as well as send a type of message to everyone in that group. Furthermore, they can list members of a room or the different rooms. Additional functionality includes private messaging, secure messaging, file transfer, and a central cloud-connected server.

The basis of this application is creating a custom protocol for which the IRC application can work correctly and effectively using TCP, IP, and Sockets. The point of this document is to outline not only the feature set and the details of their implementation, but also the custom protocol method used.

2. Basic Information

All communication outlined in this document takes place over TCP/IP. The server will have an open port (e.g., 3000) that is always listening for incoming requests from the client. Once the first request is sent from the client, it opens up a connection efficiently, starting a two-path gateway between the server and client. Both of which are free to send messages at any time.

Every message from the client to the server will warrant a response if it reaches the server successfully. Either with a successful message or an error providing some information on the problem. Either end may also terminate the connection, although primarily that task will typically be handled by the client, and will only be done by the server in the case of a timeout.

The client messages must contain information in the header on the receivers, the message type, and the message content. Even messages that are transferred as part of a stream (e.g., file transfer) must include this header information. The server's response messages will include the user ID/name of the message receiver as well as the message itself.

The client controls their actions within the server however, the server is responsible for executing the command as well as preventing unauthorized or incorrect commands. As well as enforcing the protocol laid out in this document.

3. Conventions

All hexadecimal values in this document are represented with the 0x prefix (e.g., 0x21). All name fields referenced in this document refer to a unique identifier string that is unique to a specific room or client, which is 20 characters long or less. Any field described as empty means that the field will exist, but will not contain any data.

4. Message Infrastructure

4.1. Generic Message Format

4.1.1. Message Format

```
class header:
    def __init__(self):
        self.operation_code = 0x00
        self.length = 1024
        self.target = ""
        self.sender = ""
```

```
class message:
    def __init__(self):
        self.header = header()
        self.data = bytearray()
```

4.1.2. Definitions

- `header.operation_code`: specifies the type of message's being sent that is within the messages payload
- `header.length`: specifies how many bytes of payload follow the header in this message (the header is not included in this number)
- `header.target`: is the targeted client for this message
- `header.sender`: is the name of the person who originally sent the message
- `message.header`: is how the header is stored within the main message body
- `message.data`: is a byte array that stores the bytes as a byte stream, which the client side then turns back into either a text message or a file stream, depending on the message type

4.1.3 Operation Codes (`header.operation_code`)

Code Name	Code Hex
MSG_IN_PROG	0x11
FILE_IN_PROG	0x12
USERS_IN_PROG	0x13
ROOMS_IN_PROG	0x14

4.1.4 Restrictions

- Data length must be in the range of 1 byte to 1024 bytes
- An operation message (described in [4.2]) must be sent to indicate the beginning of sending data in the general message format, and then another operation message must be

sent afterwards to signal the completion of sending the messages related to the data being transmitted

- It is up to the client to parse the byte array to either a string message or a file

4.2. Operation Messages

4.2.1. Message Format

```
class operation_message:
    def __init__(self):
        self.operation_code = 0x00
        self.data = ""
        self.target = ""
        self.sender = ""
```

4.2.2. Definitions

- `operation_message.operation_code` = the hex code of the operation this message is conveying
- `operation_message.data` = includes any necessary data that is required for the operation to occur (some operations don't need this data)
- `operation_message.target` = the targeted user or group of this operation message
- `operation_message.sender` = the individual sender of this operation message

4.2.3. Operation Codes (`operation_message.operation_code`)

Code Name	Data Requirements	Code Hex
CONNECT_TO_SERVER	Client name	0x21
CREATE_ROOM	Room name	0x22
JOIN_ROOM	Room name	0x23
LEAVE_ROOM	Room name	0x24

DISCONNECT		0x25
LIST_MEMBERS_OPEN	Room name	0x26
LIST_MEMBERS_CLOSE	Room name	0x27
LIST_ROOMS_OPEN		0x28
LIST_ROOMS_CLOSE		0x29
SEND_MSG_OPEN		0x2a
SEND_MSG_CLOSE		0x2b
SEND_FILE_OPEN	File name	0x2c
SEND_FILE_CLOSE		0x2d
SUCCESS		0x2e

4.2.4. Restrictions

- Any operations that have an open and a close are intended to send messages in between those two operations
- Any operations that have an open and close have a timeout timer of 30 seconds. If the close or a message is not received every 30 seconds, then it automatically times out and sends an error message referenced in [4.3]
- Any operation message that does not send any data will just leave that field empty

4.3. Error Messages

4.3.1. Message Format

```
class error_message:
    def __init__(self):
        self.err_code = 0x00
        self.data = ""
        self.target = ""
```

```
self.sender = ""
```

4.3.2. Definitions

- `error_message.err_code` = the hex code of the error this message is conveying
- `error_message.data` = includes data on the error message, which can often be used for displaying to the client; it may be empty if the client is sending the error
- `error_message.target` = the targeted user or group of this error message
- `error_message.sender` = the individual sender of this error message

4.3.3. Error Codes (`error_message.err_code`)

Code Name	Error Message	Code Hex
SERVER_NOT_FOUND	The server was not found	0x31
CLIENT_NOT_FOUND	The client was not found	0x32
NAME_EXISTS	This name is already taken	0x33
INVALID_NAME	This name is invalid	0x34
TIMEOUT_ERROR	Client timed out due to inactivity	0x35
INVALID_OPERATION	An invalid operation was passed in	0x36
INVALID_PROTOCOL	An incorrect protocol attempted	0x37
INVALID_ROOM	The room listed does not exist	0x38
INVALID_CLIENT	The client listed does not exist	0x39
ILLEGAL_LENGTH	The length of the data is wrong	0x3a
TOO_MANY_USERS	There are too many users	0x3b
TOO_MANY_ROOMS	There are too many rooms	0x3c
UNKNOWN_ERR	An unknown error occurred	0x3d

4.3.4. Restrictions

- Error messages do not automatically disconnect the connection between the client and server.
- Error messages are sent when an action was not correctly performed or completed, and often indicate that the action was not done.
- Any actions, such as removing half-downloaded files, are on the client to implement and complete if they are the ones receiving the error message.

4.4. Global Message Restrictions

- No group or user can be called “server.”
- All names of groups and servers must be unique and must be under 20 characters; thus, anything referencing these names will be under 20 characters.
- Every message must have one of its valid action codes.
- If any of these restrictions are broken, an error code corresponding to the issue must be sent to the client.
- If more than 100 clients are connected or more than 100 rooms are created, then the 0x3b or 0x3c will be sent to the client.

5. Client Messages

5.0.1 General Global Guidelines

- Some actions are a combination of server and client; however, the actions specified in this category are mostly client operations.
- Every action will warrant a response from the server of either success through the 0x2e operation code or an error message. If no response is received from the server, it can be assumed that the action was not completed.
- The sender field in all the definitions will be replaced with the client name, and the data will be replaced with the actual data.

- When the server is described to complete some type of functionality in the usage sections, it is assumed that this is the case in the absence of errors.
- When receiving data it will check for “TCP Message Coalescing” by checking for multiple message separated by the ‘\n’ delimiter character between messages

5.1. Connecting to the Server

5.1.1 Definition

```
msg = operation_message()  
msg.operation_code = 0x21  
msg.target = “server”  
msg.sender = “client”  
msg.data = “client_socket”
```

5.1.2 Usage

To initially connect to the server, the client will send a message to the server with the 0x21 operation code. With the sender name becoming the name the server stores for this user. The client’s socket is automatically received upon the initial connection.

5.2. Create a Room

5.2.1 Definition

```
msg = operation_message()  
msg.operation_code = 0x22  
msg.target = “server”  
msg.sender = “client”  
msg.data = “room_name”
```

5.2.2 Usage

To create a room, the client will send a message with the 0x22 operation code as well as the “room_name” to the server. The server will then create this room, unless there is an error stopping this operation.

5.3. Join a Room

5.3.1 Definition

```
msg = operation_message()  
msg.operation_code = 0x23  
msg.target = "server"  
msg.sender = "client"  
msg.data = "room_name"
```

5.3.2 Usage

To join a room, the client will send a message with the 0x23 operation code as well as the "room_name" to the server. The server will then add this user to the room.

5.4. Leave a Room

5.4.1 Definition

```
msg = operation_message()  
msg.operation_code = 0x24  
msg.target = "server"  
msg.sender = "client"  
msg.data = "room_name"
```

5.4.2 Usage

To leave a room, the client will send a message with the 0x24 operation code, as well as the "room_name" to the server. The server will then remove this user from the room.

5.5. Disconnect from the Server

5.5.1 Definition

```
msg = operation_message()  
msg.operation_code = 0x25  
msg.target = "server"
```

```
msg.sender = "client"
```

5.5.2 Usage

To disconnect from the server, the client will send a message with the 0x25 operation code to the server. The server then disconnects from the client.

5.6. Send a Message

5.6.1 Definition

```
msg = operation_message()  
msg.operation_code = 0x2a  
msg.target = "group_or_person_name"  
msg.sender = "client"
```

5.6.2 Usage

To send a message to a group or a singular person, the client first sends an operation message with the code 0x2a. Then they send the general message in one chunk or multiple chunks, depending on the size of the message. Finally, once they are done, they send another operation message to the server with the 0x2b, which indicates that they are done sending a message. The server echoes these messages to everyone specified in the original operation message. For simplicity, the closing operation message as well as the general message are not shown here, although they would be filled out with the corresponding data. Also, if the server does not receive a message or a close message operation within 30 seconds after the "stream" was opened, then it automatically times out and sends an error message to the corresponding users.

5.7. Send a File

5.7.1 Definition

```
msg = operation_message()  
msg.operation_code = 0x2c  
msg.target = "group_or_person_name"  
msg.sender = "client"
```

5.7.2 Usage

To send a file to a group or a singular person, the client first sends an operation message with the code 0x2c. Then they send the general message in one chunk or multiple, depending on the size of the file. Finally, once they are done, they send another operation message to the server with the 0x2d, which indicates that it is done sending the file. The server echoes these messages containing the files to everyone specified in the original operation message. For simplicity, the closing operation message as well as the general message are not shown here, although they would be filled out with the corresponding data. Also, if the server does not receive a message or a close message operation within 30 seconds after the “stream” was opened, then it automatically times out and sends an error message to the corresponding users.

6. Server Messages

6.0.1 General Global Guidelines

- Some actions are a combination of server and client; however, the actions specified in this category are mostly client operations.
- Every action will warrant a response from the client of either success through the 0x2e operation code or an error message. If no response is received from the server, it can be assumed that the action was not completed.
- The sender field in all the definitions will be replaced with the client name, and the data will be replaced with the actual data.
- When the server or client is described to complete some type of functionality in the usage sections, it is assumed that this is the case in the absence of errors.
- When sending data the server will prevent “TCP Message Coalescing” by adding a ‘\n’ delimiter character after each message

6.1. List Members

6.1.1. Definition

```
msg = operation_message()  
msg.operation_code = 0x27  
msg.target = "client"  
msg.sender = "server"
```

6.1.2. Usage

After receiving a list of members' open operations, the server then begins to send back general messages containing the members. Once all the members have been sent, the server sends back a 0x27 operation, which indicates to the client that the "stream" has been closed and all the members have been sent. For simplicity, only the close list members are shown in the definition.

6.2. List Rooms

6.2.1. Definition

```
msg = operation_message()  
msg.operation_code = 0x29  
msg.target = "client"  
msg.sender = "server"
```

6.2.2. Usage

After receiving a list of open rooms, the server then begins to send back general messages containing the rooms. Once all the rooms have been sent, the server sends back a 0x29 operation, which indicates to the client that the "stream" has been closed and all the rooms have been sent. For simplicity, only the closed list of rooms is shown in the definition.

6.3. Success

6.3.1. Definition

```
msg = operation_message()  
msg.operation_code = 0x2e
```

```
msg.target = "client"
```

```
msg.sender = "server"
```

6.3.2. Usage

In the case where something is successful but no need for a response is required, the server will send a simple 0x2e success response. Indicating to the client that whatever was sent was successfully received.

7. Error Handling

Both the server and the client are expected to detect errors. Thus, in the code, both applications will have robust error handling that falls into one of the above-mentioned error classes. In the case of an error, either the client or server will send an error message to the other. Because after every message, there is an expected response, errors should not easily be detected unless the message is lost. However, to combat this flaw, there will be an expected response time for every action. Through this documentation, those times are specified; and if a message is not received within that time frame, another error message will be sent, which again expects a response.

8. Extra Functionality

8.0. Extra functionality foreword

The descriptions below provide a high-level overview of how these extra functionality features can be implemented. The architecture outlined in this document is designed to support all of these features. These descriptions are planned to aid a reader in understanding how they fit into the already existing architecture described in length in this document.

8.1. Private Messaging

To allow for private messaging, the server prevents room names and client names that are the same. Thus, when a name is specified, it is searched for in both the room names and the client names. And is delivered to the respective user. This does not introduce a large overhead since both the client count and room count are capped at 100, so at maximum capacity, it will not exceed 200 searches.

8.2. Secure messaging

This feature requires no change to the structure of the messages. It only requires each end to use the respective library that does encryption. Then, for the server to decrypt it, it encrypts it again before passing on the message to another client, which can decrypt the message using the same library. No further architecture needs to be implemented to accommodate this.

8.3. File Transfer

To accommodate file transfer, an operation message was added to specifically indicate that files are being transferred instead of text. Furthermore, in the general message structure, the data is a byte array rather than a string, allowing for any data, whether it be file data or strings, to be transferred. Then the receiving client, given that they received the original operational message, can correctly identify what was sent.

9. Security

Security in this IRC application is being dealt with by the encryption that will be implemented. This encryption uses a library that is guaranteed to provide safe and secure encryption. If the message is intercepted by any onlooker, the message will be encrypted and practically uncrackable.

10. Conclusion and Future Directions

This document provides clear and in-depth documentation of the IRC application and its protocol. It will allow a reader to understand not only the architecture but also how it functions and how it contributes to the feature set of this IRC application.

In the future, this architecture will allow for many new features. And if changes need to be attempted was made to make the design modular so changes to the protocol can be fairly easy to implement. Although this protocol is specifically designed for a certain set of features and applications, its use in other applications is not entirely guaranteed